Evaluating Graph-Based and Standard Machine Learning Approaches for Airbnb Pricing Predictions

Team Members: Ethan Cohen, Haotian (Owen) Cui, Matthieu Hautsch

Emails: ethancoh@stanford.edu, owencui@stanford.edu, matthaut@stanford.edu

1 Motivation

This project was inspired by three international first-year master's students who share a passion for traveling. While exchanging travel stories, we noticed that Airbnb consistently emerged as the most popular accommodation choice, which led us to ask: What metrics allow Airbnb to fairly and accurately determine a listing's price?

Through research, we discovered that short-term rental pricing depends not only on listing-level attributes such as capacity, room type, amenities, and textual description, but also on broader neighborhood effects, including proximity to public transit, nightlife, and local attractions.

In this project, we frame the problem as a supervised regression task aimed at predicting Airbnb listing prices using a combination of machine learning models. Ultimately, our goal is to model the city as a graph, where each listing is represented as a node and edges connect geographically or semantically similar listings. Beyond academic interest, this work has practical value for the short-term rental industry by offering a modeling framework that captures the complex interplay between spatial, textual, and tabular data in price formation.

2 Methods

2.1 Data Preprocessing

The San Francisco Airbnb dataset (1) was preprocessed to focus on short-term stays and relevant features, including location, room characteristics, amenities, reviews, and host information. Prices were cleaned, extreme values were capped, and a **log transformation** was applied to the target variable (`log_price`) to normalize its distribution and mitigate right skewness (see **Figure 1** and **Figure 2** in **Appendix B**). Additional features were engineered to capture listing comfort (`#bedrooms`, `#bathrooms`, `#beds`) and the number of amenities. Exploratory data analysis was performed, including correlation matrices, price distribution visualizations, and neighborhood-level price summaries, to understand patterns and relationships in the data.

2.2 Train/Test Splitting

Initially, we decided to randomly split the dataset into training and testing sets. However, after further discussion, we realized that random splitting can be misleading, as geographically close listings may appear in both sets, artificially inflating model performance. To address this issue, we applied k-means clustering on latitude and longitude to group listings into distinct geographical regions. Specifically, we clustered a total of 5,182 listings into 20 geographically coherent clusters and then randomly selected 5 clusters to form the testing set. This resulted in 4,402 listings (approximately 84.95%) for training and 780 listings (approximately 15.05%) for testing. A visualization of the resulting train-test split is shown in **Figure 3** (see **Appendix B**).

To verify that random splitting would not yield an honest measure of spatial generalization, we ran the same algorithms with equivalent settings under two schemes: (i) standard 5-fold random split (geo-unaware) and (ii) leave-location-out/ k-means clustering cross-validation using geoclusters (geo-CV with 5 folds, **Figure 3**). As expected, training MAE is lower in both schemes because models are graded on data they have seen (**Table 1**). For Random Forest and XGBoost, we can see a lower training MAE for the geo-aware split. It is expected because the model is being graded on the exact data it practiced on. Algorithms like XGBoost can learn very specific patterns from those listings (even neighborhood-specific quirks), so it partly "memorizes" them and drives train error down. When we switch to geo-CV, it faces new areas it never saw before, those shortcuts don't transfer, and the error goes up, exactly what honest generalization looks like. Crucially, geo-CV errors are higher than random-CV errors across all models, which is exactly what honest spatial generalization looks like: when held-out areas are truly unseen, neighborhood-specific shortcuts

learned during training no longer transfer. If geo-CV matched random CV, we would conclude proximity adds little. Since geo-CV is worse, **geography clearly matters**; therefore, we report geo-CV as the primary metric and it confirms it is relevant to upgrade our modeling to capture spatial structure with GNN.

Table 1: Same Algorithms Under Two Evaluation Schemes

	Random 5-fold CV (geo-unaware)				Leave-location-out Geo-CV			
Model	**MAE (Train)**	**RMSE (Train)**	**MAE (5-fold CV)**	**RMSE (5-fold CV)**	**MAE (Train)**	**RMSE (Train)**	**MAE (5-fold geo-CV)**	**RMSE (5-fold geo-CV)**
Linear Regression	0.350	0.452	0.350	0.452	0.355	0.453	0.416	0.519
Decision Trees	0.289	0.382	0.342	0.466	0.300	0.395	0.386	0.505
Random Forest	0.101	0.142	0.276	0.386	0.099	0.139	0.347	0.452
XGBoost	0.115	0.159	0.270	0.369	0.102	0.135	0.353	0.455

2.3 Machine Learning Models (Implemented so far)

Linear Regression (Baseline): Linear Regression is selected as the baseline model. Since the target variable is the logarithm of price, this model effectively captures multiplicative relationships between features and price. Its simplicity, interpretability, and computational efficiency make it an ideal starting point and a benchmark for comparing more complex models.

LASSO & Ridge: LASSO and Ridge Regression extend the baseline linear model by introducing regularization to prevent overfitting and handle multicollinearity among correlated features. LASSO adds an L1 penalty that encourages sparsity and performs feature selection, while Ridge applies an L2 penalty that shrinks coefficients smoothly toward zero. The regularization strength, controlled by the hyperparameter λ, is tuned during cross-validation (described later) to balance model complexity and predictive accuracy

Decision Trees: Decision Trees are implemented to capture nonlinear relationships and feature interactions that linear models cannot represent. They recursively partition the feature space based on split criteria that minimize prediction error, allowing for flexible, interpretable decision boundaries. To control overfitting, the pruning parameter ccp_alpha (cost-complexity pruning coefficient) is tuned as a hyperparameter

Random Forest: Random Forests are employed to enhance prediction stability and capture complex nonlinear relationships by averaging the outputs of multiple decision trees trained on random subsets of data and features. This ensemble approach reduces variance and mitigates overfitting. It includes several key hyperparameters, such as the number of trees and the maximum tree depth, which are tuned to balance model complexity and generalization

XGBoost: XGBoost is implemented as a gradient-boosted tree model that sequentially builds decision trees, with each tree correcting the residual errors of the previous ones. This boosting framework allows the model to capture complex nonlinear relationships while maintaining high predictive accuracy. Compared to Random Forests, XGBoost emphasizes bias reduction through weighted tree updates and regularization. Key hyperparameters such as the learning rate, maximum depth, and number of estimators are tuned to optimize performance on log-transformed Airbnb prices.

K-Nearest Neighbors (KNN): KNN is used to explore local similarity patterns in Airbnb pricing, based on the assumption that listings with similar features and nearby locations tend to have comparable prices. The model predicts each listing's price as the average of its k nearest neighbors in feature space. As a non-parametric method, KNN can capture localized nonlinear trends but is sensitive to feature scaling and the choice of k, which is tuned during cross-validation to achieve optimal performance on log-transformed prices.

2.4 Model Evaluation

A 5-fold geographical cross-validation approach is applied to ensure that model evaluation reflects spatial generalization rather than random overlap. Instead of randomly splitting the data, listings are split according to their assigned geographical cluster labels, with each fold holding out one cluster group for validation while training on the remaining clusters. This method mirrors the spatial train-test split and provides a more realistic assessment of how models perform on unseen regions. The same procedure is used both for hyperparameter tuning and for estimating test error, ensuring consistent and unbiased performance evaluation.

3 Preliminary Experiments and Results

The performance of each model is evaluated using **Mean Absolute Error (MAE)** and **Root Mean Squared Error (RMSE)** on the log-transformed prices. MAE measures the average absolute deviation between predicted and true log prices, while RMSE penalizes larger errors more heavily due to the square term. To interpret MAE in the log scale, we exponentiate it to obtain $\exp(\text{MAE}_{\log})$, which represents the *geometric mean of the multiplicative error factors* between predicted and true prices. For instance, an $\text{MAE}_{\log} = 0.35$ corresponds to an average multiplicative error of $e^{0.35} \approx 1.42$, meaning that model predictions differ from actual prices by roughly $\pm 42\%$ on average. (See **Appendix A** for the full derivation.)

Table 2 presents the training and 5-fold geo-based cross-validation performance for all models, while **Table 3** (**Appendix C**) lists the corresponding hyperparameters. As expected, the cross-validation errors are consistently higher than the training errors, reflecting the more challenging task of generalizing to unseen geographical regions. Ensemble models such as Random Forest and XGBoost achieve the lowest cross-validation MAE and RMSE, leveraging their ability to average over multiple trees and capture complex nonlinear feature interactions.

Table 2: Model Performance Comparison

Model Name	MAE (5-fold geo-CV)	RMSE (5-fold geo-CV)
Linear Regression	0.416	0.519
LASSO	0.412	0.516
Ridge	0.412	0.515
Decision Trees	0.386	0.505
Random Forest	0.345	0.449
XGBoost	0.346	0.446
KNN	0.437	0.569

4 Next Steps

Based on our preliminary experiments, including random and geographically-aware splits, we observed that model performance drops when predicting prices in unseen areas. This highlights the importance of capturing spatial dependencies. To address this, we plan to implement a **Graph Neural Network (GNN)**, representing listings as nodes connected by spatial and textual similarity. We will explore architectures such as **GraphSAGE** or **GCN**, with careful hyperparameter tuning (e.g., number of layers, neighborhood aggregation size, dropout) to optimize performance. This approach follows recent work by Kanakaris and Karacapilidis (2), who successfully applied GNNs combined with document embeddings to predict Airbnb listing prices on the island of Santorini. We also plan to enhance features by incorporating textual information from listing descriptions. This will be done using embeddings generated from LLM-based models or traditional approaches like **Doc2Vec**, enabling the model to capture semantic information. All previous models will be retrained with these enriched features to allow a fair comparison.

Model selection will rely on **5-fold cross-validation MAE**, and the final performance will be assessed on a held-out test set. Additional analyses of feature importance and spatial performance will ensure that the models generalize effectively across neighborhoods. Finally, scalability considerations will be addressed to ensure that the methodology can be applied beyond San Francisco or to larger datasets.

5 Team Contributions

The team meets regularly to work on all aspects of the project, including but not limited to data preprocessing, model training, geo-based cross-validation, and report writing. Each team member contributes evenly, with individuals taking the lead in different areas:

- **Ethan**: Led the geo-aware clustering and train-test split design, and planned the next steps.
- **Owen**: Led data preprocessing, feature engineering, and the setup of cross-validation.
- **Matthieu**: Led the implementation and hyperparameter tuning of various ML models.

References

[1] Inside Airbnb - San Francisco Detailed Listings data

[2] Nikos Kanakaris & Nikos Karacapilidis, 2023, *Predicting prices of Airbnb listings via Graph Neural Networks and Document Embeddings: The case of the island of Santorini Procedia Computer Science*, Volume 219, 2023, Pages 705-712.

A Derivations

Interpretation for MAE and RMSE

Let the true price be y_i and the predicted price be $\hat{y}_i$. Define the true log price as $\ell_i = \log(y_i)$ and the predicted log price as $\hat{\ell}_i = \log(\hat{y}_i)$.

Let the error ratio be:

$$r_i = \frac{y_i}{\hat{y}_i}$$

Then:

$$\ell_i - \hat{\ell}_i = \log(y_i) - \log(\hat{y}_i) = \log\left(\frac{y_i}{\hat{y}_i}\right) = \log(r_i)$$

Define:

$$f_i = e^{|\log(y_i/\hat{y}_i)|} = e^{|\log r_i|} = e^{|\ell_i - \hat{\ell}_i|}$$

Thus, the Mean Absolute Error in log space is:

$$\text{MAE}_{\log} = \frac{1}{n}\sum_{i=1}^{n} |\ell_i - \hat{\ell}_i| = \frac{1}{n}\sum_{i=1}^{n} \log(f_i)$$

Exponentiating both sides gives:

$$\exp(\text{MAE}_{\log}) = \exp\left(\frac{1}{n}\sum_{i=1}^{n} \log(f_i)\right)$$

By properties of the exponential and logarithm:

$$\exp(\text{MAE}_{\log}) = \left(\prod_{i=1}^{n} f_i\right)^{1/n}$$

Hence, $\exp(\text{MAE}_{\log})$ represents the **geometric mean of the multiplicative error factors** f_i.

B Additional Figures

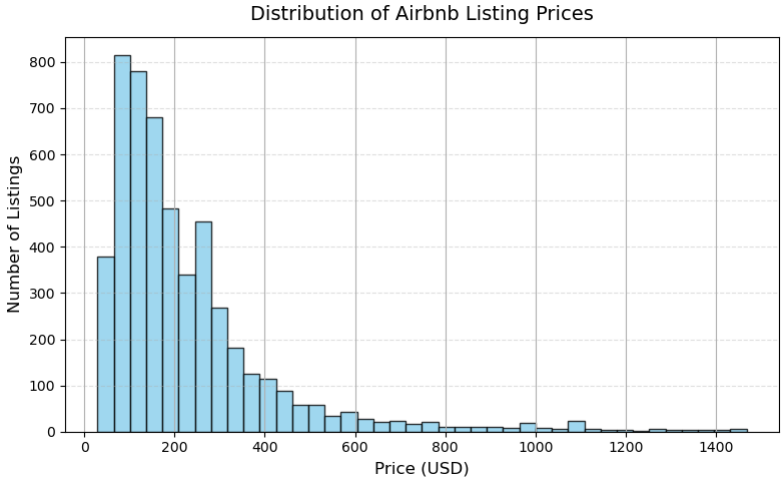

Figure 1: Distribution of Airbnb Listing Prices

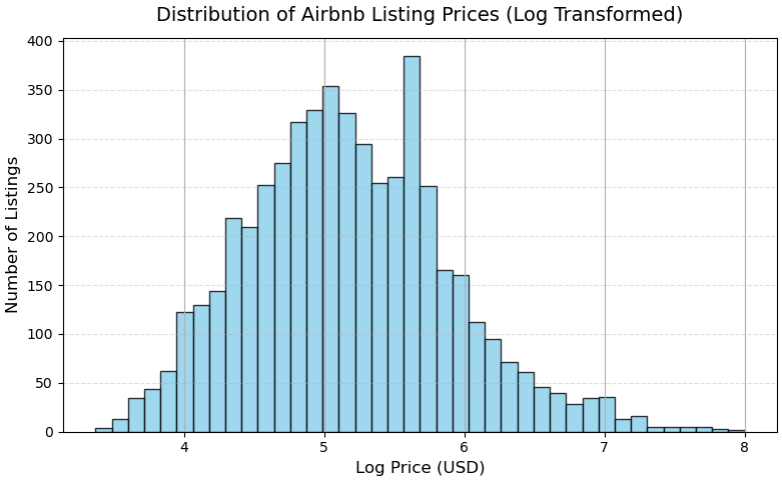

Figure 2: Distribution of Airbnb Listing Price (Log Transformed)

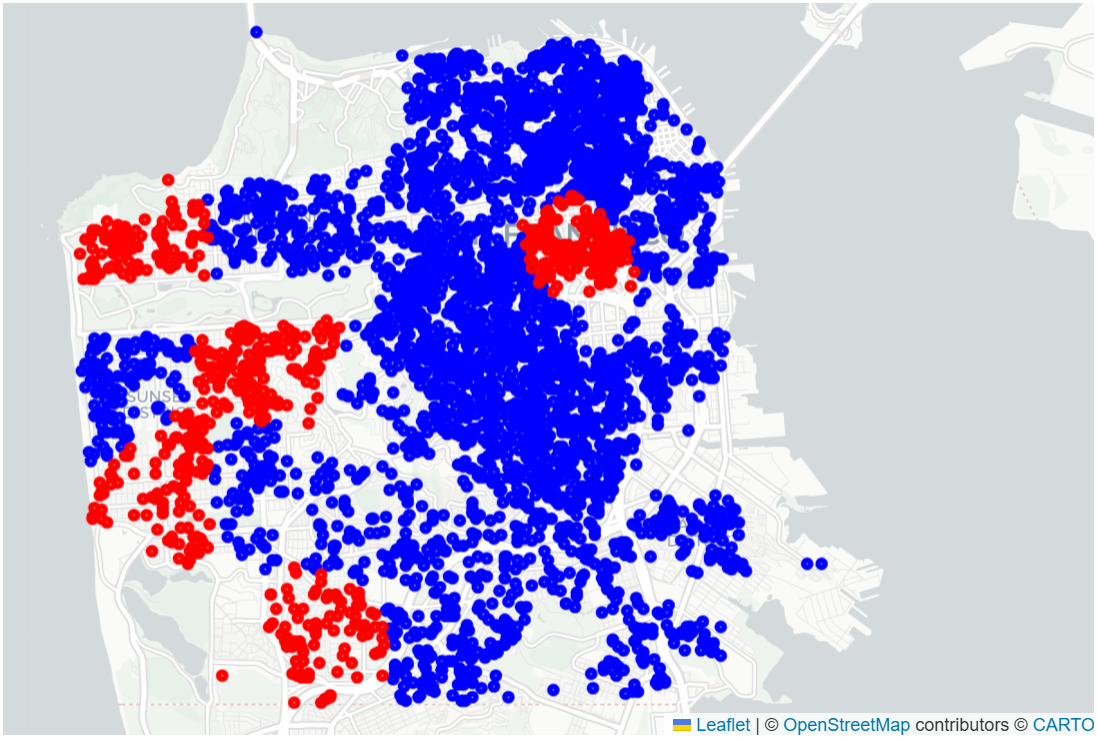

Figure 3: Geographical Train-Test Split of San Francisco Airbnb Listings

C Additional Tables

Table 3: Model Hyperparameter Summary

Model Name	**Hyperparameters**
Linear Regression	Default (no hyperparameter regularization)
LASSO	$\lambda = 0.00010$
Ridge	$\lambda = 0.01984$
Decision Trees	ccp_alpha = 0.00076
Random Forest	n_estimators = 400, max_depth = 22, min_samples_split = 2, min_samples_leaf = 1
XGBoost	learning_rate = 0.0164, max_depth = 5, n_estimators = 307, subsample = 0.838, colsample_bytree = 0.885, reg_λ = 2.357, reg_α = 0.0117
KNN	n_neighbors = 18, weights = "uniform", p = 2, n_components = 5